

Auditoria de Consistencia - ERP y Pizarra Inteligente

- [AUDITORÍA DE CONSISTENCIA CRUZADA — ERP ↔ PIZARRA INTELIGENTE](#)
 - [Qué se contradice, por qué pasó, y la corrección de raíz \(no un parche cosmético\)](#)
 - [HALLAZGO 1 — NOMBRES DE CARPETA PYTHON INVÁLIDOS \(CONTRADICCIÓN REAL, NO ESTILO\)](#)
 - [HALLAZGO 2 — EL “CONFLICT-RESOLUTION AGENT” HACÍA TRES TRABAJOS DISTINTOS BAJO UN SOLO NOMBRE](#)
 - [GATES NUEVOS QUE EVITAN QUE ESTO VUELVA A PASAR](#)
 - [POR QUÉ ESTO SÍ ES “OTRO NIVEL” Y NO UN AJUSTE COSMÉTICO](#)

AUDITORÍA DE CONSISTENCIA CRUZADA — ERP ↔ PIZARRA INTELIGENTE

Qué se contradice, por qué pasó, y la corrección de raíz (no un parche cosmético)

Este documento audita ERP_ENTERPRISE_AUTONOMO_V2_ROBUSTO.md (V1-V10) y PIZARRA_INTELIGENTE_ALTA_INGENIERIA.md (V1-V5) entre sí, ahora que la segunda declara reutilizar módulos de la primera. Cada hallazgo sigue el mismo formato: **evidencia citada** → **causa raíz** → **corrección aplicada**, no una reformulación superficial.

HALLAZGO 1 — NOMBRES DE CARPETA PYTHON INVÁLIDOS (CONTRADICCIÓN REAL, NO ESTILO)

Evidencia

El árbol de archivos del ERP declara estas carpetas con **guion**:

```
backend/app/modules/voice-assistant/  
backend/app/modules/integrations-social/  
backend/app/modules/integrations-social/meta-webhooks/  
backend/app/modules/field-operations/
```

Pero **todo el código Python** de las adendas V7-V9 las importa con **guion bajo**:

```
from app.modules.voice_assistant.confidential_guard.permission_check import ... # V7  
from app.modules.voice_assistant.confidential_guard.redaction_rules import ... # V9  
from app.modules.integrations_social.token_vault import ... # V8  
from app.modules.integrations_social.meta_webhooks.idempotency_store import ... # V9
```

Causa raíz

Python **no permite** import de un paquete cuyo directorio tiene un guion (-) en el nombre — el guion es el operador de resta y rompe la sintaxis del import (import app.modules.voice-assistant es un error de sintaxis, no un error de “módulo no encontrado”). Cada vez que agregué una adenda nueva (V3, V4, V5) copié el estilo de nombres del árbol original del PDF (que usaba guion para *todo*, incluyendo carpetas de docs/ donde sí es válido), y cuando llegué a escribir código Python real (V7 en adelante) mi propio intérprete interno de Python me obligó a usar guion bajo — sin volver atrás a corregir el árbol. El documento se contradice a sí mismo porque nunca hubo una regla explícita de “esto es Python, esto es solo documentación”, y ambas convenciones convivieron sin que nadie las conciliara.

Corrección de raíz

Se establece una regla única, no ambigua, y se agrega como **gate automatizado** (no solo como buena intención):

```
# agents/policies/naming_convention.yaml + CORRECCIÓN
rule: "Toda carpeta bajo backend/app/, agents/runtime/, agents/specialists/ y firmware/
que sea un paquete Python (contiene __init__.py o se importa con `from`/`import`)
usa snake_case (guion_bajo). El guion (-) está PROHIBIDO en esas rutas.
Las carpetas bajo docs/, hardware/, .github/, y nombres de recurso no-Python
(ej. slugs de URL, nombres de artefacto) Sí pueden usar guion."
enforcement: "pre_tool.py (hook de Claude Code, ya existente en .claude/hooks/)
rechaza cualquier `create_file` o `bash mkdir` que cree una carpeta con guion
bajo backend/app/modules/, agents/runtime/, agents/specialists/ o firmware/."
```

Árbol corregido (reemplaza las 4 rutas de la sección 1 del ERP y sus referencias en V3/V4/V5/V8/V9):

Antes (roto)	Después (corregido, coincide con el código ya escrito)
backend/app/modules/voice-assistant/	backend/app/modules/voice_assistant/
backend/app/modules/integrations-social/	backend/app/modules/integrations_social/
backend/app/modules/integrations-social/meta-webhooks/	backend/app/modules/integrations_social/meta_webhooks/
backend/app/modules/field-operations/	backend/app/modules/field_operations/

No se corrigió el código Python (V7-V9) porque **ya estaba bien** — el guion bajo es la forma correcta; lo que estaba mal era el árbol de carpetas de las secciones 1, 21, 22 y 24, que ahora queda alineado con el código real en vez de al revés.

Referencias en docs/, .claude/agents/*.md, nombres de integración (integrations-social como concepto de negocio) **se mantienen con guion** cuando son prosa o rutas de documentación, no import de Python — ahí el guion es válido y más legible.

HALLAZGO 2 — EL “CONFLICT-RESOLUTION AGENT” HACÍA TRES TRABAJOS DISTINTOS BAJO UN SOLO NOMBRE

Evidencia

- En el ERP (V10, §48), el Conflict-Resolution Agent se define como agente de **Claude Code en tiempo de construcción** — arbitra cuando “dos o más agentes producen cambios incompatibles... sobre el mismo ciclo del loop autónomo (DISCOVER→...→COMMIT)”. Es decir: agentes de IA escribiendo código durante el desarrollo del ERP.
- Su implementación (§49) se ubicó en agents/runtime/orchestrator/conflict_resolver.py, que según el árbol de la sección 1 del ERP es la carpeta del **sistema de agentes de producción del ERP mismo** (el módulo ai/ de negocio, orquestación de procesos como aprobaciones, no la construcción del software).
- La Pizarra (V1, §8) reutiliza ese mismo archivo para resolver que **dos alumnos humanos** conviertan la misma zona del lienzo a la vez — un tercer contexto completamente distinto: concurrencia de usuarios finales en una aplicación educativa, sin ningún agente de IA de por medio.
- Al importarlo, la Pizarra usa una ruta (app.modules.agents_runtime.orchestrator.conflict_resolver) que **no coincide** ni con la ruta original del ERP (agents/runtime/orchestrator/conflict_resolver.py, fuera de backend/app/) ni con ninguna convención usada en el resto de los documentos.

Causa raíz

Confundí **tres capas que no comparten ciclo de vida, ni límite de confianza, ni siquiera quién las ejecuta**:

Capa	Quién la ejecuta	Cuándo corre	Qué protege
Build-time (Claude Code)	Los propios agentes de IA que construyen el software	Solo durante desarrollo/mantenimiento autónomo	Que dos agentes no se pisen editando el repositorio
Runtime de negocio (agentes del producto)	El backend del ERP en producción	Durante uso normal del ERP por sus clientes	Que dos procesos de negocio automatizados no tomen decisiones contradictorias
Concurrencia de usuario final	El backend de cualquier producto (ERP o Pizarra) atendiendo humanos reales	Durante el uso normal por personas	Que dos personas no pisen la misma edición al mismo tiempo

Reutilizar el mismo archivo para las tres fue un error de diseño real: mezclé una herramienta de *tooling de desarrollo* (que ni siquiera se despliega al cliente final) con una necesidad genérica de *locking distribuido* que sí es legítimamente compatible entre productos — pero no bajo el nombre ni la ubicación de un “agente”.

Corrección de raíz — separar en tres componentes reales

a) El Conflict-Resolution Agent (Claude Code, build-time) se queda exactamente donde estaba — `.claude/agents/conflict-resolution.md` + su propia implementación de soporte, pero corrigiendo que NO viva dentro de `backend/app/` (no es parte del producto que se despliega a un cliente, es *tooling de construcción*):

```
# agents/runtime/orchestrator/conflict_resolver.py    + CORREGIDO
# Ya NO importa desde app.db.session (ese módulo pertenece al backend
# desplegable, agents/ es un proceso de tooling separado que corre
# solo durante sesiones de Claude Code).
from agents.runtime.db import get_tooling_redis # + conexión propia, mismo REDIS_URL por
variable de entorno

# ... el resto de acquire_lock/release_lock/detect_contract_drift
# ($49 del ERP) se mantiene sin cambios de lógica, solo la fuente de
# la conexión Redis cambia de "compartida con el backend" a "propia
# del proceso de tooling".

# agents/runtime/db.py    + NUEVO – antes no existía, causaba el cruce indebido
import os
import redis.asyncio as redis

def get_tooling_redis() -> redis.Redis:
    # Mismo REDIS_URL de infraestructura que el backend (una sola
    # instancia de Redis física), pero una conexión lógicamente propia
    # del proceso de agentes de construcción – nunca importa código
    # de backend/app/, para que agents/ pueda desplegarse o incluso
    # dejar de ejecutarse sin afectar el producto en producción.
    return redis.from_url(os.environ["REDIS_URL"], decode_responses=True)
```

b) Se extrae una librería genérica de locking distribuido, sin nombre de “agente”, para lo que de verdad es un problema compartido entre productos (aprobaciones concurrentes en el ERP, zonas del lienzo en la Pizarra):

```
# backend/app/platform/concurrency/resource_lock.py    + NUEVO
"""
Locking distribuido genérico (Redis SETNX + TTL). Es la MISMA técnica
que ya usaba conflict_resolver.py, pero como librería de plataforma
reutilizable por cualquier módulo de negocio – sin implicar que haya
un "agente" de por medio. El ERP y la Pizarra dependen de ESTO, no
uno del tooling de desarrollo del otro.
"""

from __future__ import annotations
from dataclasses import dataclass
from datetime import timedelta
from app.db.session import get_redis
```

```

DEFAULT_LOCK_TTL = timedelta(seconds=1800)

@dataclass(frozen=True)
class ResourceLockRequest:
    holder_id: str # user_id, o cualquier identificador del proceso que pide el lock
    resource_key: str # ej. 'invoice.approval.4521' (ERP) o 'whiteboard.room42.zone.'
(Pizarra)

class ResourceLockConflictError(Exception):
    def __init__(self, resource_key: str, held_by: str):
        super().__init__(f'{resource_key}' bloqueado por '{held_by}')
        self.resource_key, self.held_by = resource_key, held_by

async def acquire_resource_lock(req: ResourceLockRequest, ttl: timedelta = DEFAULT_LOCK_T
None:
    redis = get_redis()
    key = f"reslock:{req.resource_key}"
    ok = await redis.set(key, req.holder_id, nx=True, ex=int(ttl.total_seconds()))
    if not ok:
        held_by = await redis.get(key)
        raise ResourceLockConflictError(req.resource_key, held_by or "desconocido")

async def release_resource_lock(req: ResourceLockRequest) -> None:
    redis = get_redis()
    key = f"reslock:{req.resource_key}"
    current = await redis.get(key)
    if current == req.holder_id:
        await redis.delete(key)

```

c) La Pizarra ya no depende del tooling de Claude Code — se corrige su import para usar la librería de plataforma real:

```

# backend/app/modules/whiteboard/collaboration/zone_conflict_bridge.py  ¡ CORREGIDO
from app.platform.concurrency.resource_lock import (
    acquire_resource_lock, release_resource_lock,
    ResourceLockRequest, ResourceLockConflictError,
)

async def convert_zone_to_shape(*, room_id: str, zone_id: str, user_id: str, shape_type: str):
    lock_request = ResourceLockRequest(
        holder_id=f"user:{user_id}",
        resource_key=f"whiteboard.{room_id}.zone.{zone_id}",
    )
    try:
        await acquire_resource_lock(lock_request)
    except ResourceLockConflictError as exc:
        raise WhiteboardZoneBusyError(
            f"Otro usuario ({exc.held_by}) ya está convirtiendo esta zona"
        ) from exc
    try:
        pass # ... aplica la conversión sketch-to-vector (documento Pizarra, §4)
    finally:
        await release_resource_lock(lock_request)

```

d) El ERP también migra su propio uso de locks de negocio (ej. doble aprobación de factura, mencionado como caso 5 en el análisis V7 del ERP) a resource_lock.py en vez de al módulo de agentes — separación limpia entre “herramienta para construir software” y “función del producto”.

GATES NUEVOS QUE EVITAN QUE ESTO VUELVA A PASAR

Gate	Nombre	Evidencia requerida
G45	Convención de nombres de paquete	Ningún directorio bajo backend/app/, agents/runtime/, agents/specialists/ o firmware/ contiene un guion — verificado por pre_tool.py

G46

Separación tooling/producto

antes de que cualquier agente cree la carpeta

agents/runtime/ (Claude Code)
nunca importa de backend/app/
ni viceversa; la única forma
de compartir una capacidad
entre ambos es una librería
explícita en
backend/app/platform/ (como
resource_lock.py), nunca un
import cruzado directo

POR QUÉ ESTO SÍ ES “OTRO NIVEL” Y NO UN AJUSTE COSMÉTICO

No fue “cambiar un nombre” — fue encontrar que el documento tenía una **confusión de arquitectura real**: un componente que se presentaba como “un agente de IA que resuelve conflictos” en realidad necesitaba ser, en dos de sus tres usos, una librería de infraestructura sin ninguna IA de por medio. Nombrarlo “agente” en los tres contextos ocultaba que el ERP y la Pizarra no debían depender del *tooling de construcción del software* para funcionar en producción — eso sí hubiera sido un fallo real: un cliente con el ERP desplegado dependiendo, sin saberlo, de que el proceso de Claude Code (que ni siquiera corre en su entorno de producción) estuviera disponible.

Con esta corrección: agents/runtime/ es exclusivamente tooling de construcción (nunca se despliega al cliente), backend/app/platform/concurrency/resource_lock.py es la única pieza compartida real entre ERP y Pizarra, y los nombres de carpeta ya no se contradicen con el código que los usa.